

ITS Manager

Documentazione Tecnica Completa

Gestionale Didattico Integrato per Istituti ITS

Architettura Completa, Stack Tecnologico, Modello Dati, API REST, DevOps e Sicurezza

Versione Documento: 1.0
Data di Rilascio: 29 Settem-
bre 2026
Autore: Team di Sviluppo ITS Manager

Stato: Completo per Presentazione Aziendale
Target Users: Segreteria, Studenti, Docenti

Contents

1	Introduzione	3
1.1	Obiettivi del Progetto	3
1.2	Target Users	3
2	Architettura del Sistema	3
2.1	Architettura Generale	4
2.2	Architettura a Microservizi	4
2.3	Communication Pattern	4
3	Stack Tecnologico	5
3.1	Frontend Stack	5
3.2	Backend Stack	5
3.3	Database Stack	5
3.4	DevOps Stack	5
4	Componenti Backend	5
4.1	FastAPI Application Structure	6
4.2	API Endpoints Principali	7
4.3	Business Logic Layer	7
4.3.1	Authentication System	7
4.3.2	Validation Layer	8
5	Componenti Frontend	8
5.1	Frontend Structure	8
5.2	Frontend Architecture	8
5.2.1	CSS Architecture	8
5.2.2	JavaScript Architecture	8
5.2.3	Responsive Design	9
6	Database e Modelli Dati	9
6.1	Schema Database - Tabelle Principali	9
6.2	Relazioni Database	9
6.3	Trigger e Constraints	9
7	Sicurezza e Autenticazione	10
7.1	Authentication Flow	10
7.2	Security Measures	10
7.2.1	Backend Security	10
7.2.2	Frontend Security	10
7.2.3	QR Code Security	10
8	DevOps e Deployment	11
8.1	Docker Configuration	11
8.1.1	Backend Dockerfile	11
8.1.2	Frontend Dockerfile	11
8.1.3	Docker Compose	11
8.2	Deployment Strategy	11
8.2.1	Development	11
8.2.2	Production	11

9	Funzionalità Principali	12
9.1	Area Segreteria	12
9.1.1	Gestione Corsi	12
9.1.2	Gestione Edizioni	12
9.1.3	Calendario Didattico	12
9.1.4	Gestione Presenze	12
9.2	Area Studente	12
9.2.1	Dashboard	12
9.2.2	Lezioni	13
9.2.3	Timbrature QR Code	13
9.3	Sistema Timbratura QR Code	13
9.3.1	Workflow	13
9.3.2	Sicurezza QR	13
10	Performance e Scalabilità	13
10.1	Performance Optimization	13
10.1.1	Frontend	13
10.1.2	Backend	14
10.1.3	Database	14
10.2	Scalability Strategy	14
10.2.1	Horizontal Scaling	14
10.2.2	Vertical Scaling	14
11	Conclusioni e Sviluppi Futuri	14
11.1	Conclusioni	14
11.2	Future Enhancements	15
11.2.1	Planned Features	15
11.2.2	Technical Improvements	15

1. Introduzione

ITS Manager è un gestionale didattico completo per istituti di formazione tecnica superiore (ITS), progettato per gestire corsi, edizioni, studenti, docenti, presenze e calendario didattico in modo integrato ed efficiente.

1.1. Obiettivi del Progetto

- Gestione centralizzata di corsi ed edizioni.
- Tracciamento presenze studenti e docenti.
- Calendario didattico interattivo.
- Sistema di timbratura via QR code.
- Dashboard analitiche per studenti e segreteria.
- Interfaccia responsive mobile-first.

1.2. Target Users

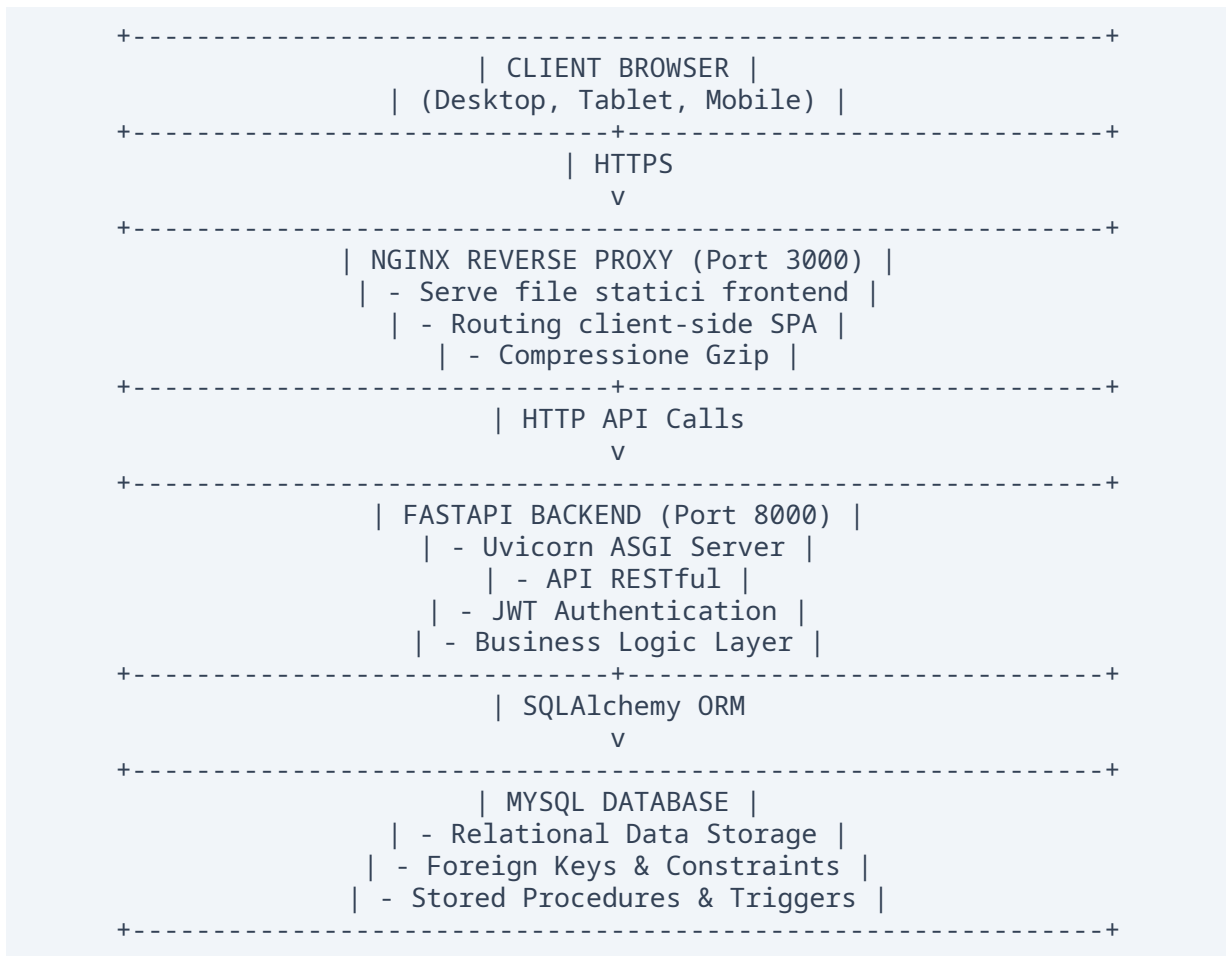
Segreteria Didattica: Gestione completa dell'istituto.

Studenti: Visualizzazione corsi, lezioni e presenze.

Docenti: Gestione lezioni e registrazioni presenze.

2. Architettura del Sistema

2.1. Architettura Generale



2.2. Architettura a Microservizi

Il sistema adotta un'architettura modulare containerizzata:

1. **Frontend Container:** NGINX + HTML/CSS/JS statici.
2. **Backend Container:** FastAPI + Python logic.
3. **Database Container:** MySQL (esterno in produzione).

2.3. Communication Pattern

- **Frontend** → **Backend:** REST API (JSON over HTTP).
- **Backend** → **Database:** SQLAlchemy ORM.
- **Authentication:** JWT Bearer Tokens + HttpOnly Cookies.

3. Stack Tecnologico

3.1. Frontend Stack

Tecnologia	Versione	Utilizzo
HTML5	-	Struttura semantica delle pagine
CSS3	-	Styling e layout responsive
JavaScript (ES6+)	-	Logica client-side
Bootstrap 5	5.x	Framework UI responsive
Bootstrap Icons	1.x	Icone vettoriali
FullCalendar	6.1.10	Calendario interattivo
QRCode.js	1.5.3	Generazione QR code
Google Fonts	Inter	Tipografia moderna

3.2. Backend Stack

Tecnologia	Versione	Utilizzo
Python	3.10	Linguaggio principale
FastAPI	Latest	Framework API REST
Uvicorn	Latest	Server ASGI
SQLAlchemy	Latest	ORM Database
PyMySQL	Latest	Driver MySQL
PyJWT	Latest	Gestione JWT
python-jose	Latest	Crittografia JWT
passlib	Latest	Hashing password
Pydantic	Latest	Validazione dati

3.3. Database Stack

Tecnologia	Versione	Utilizzo
MySQL	8.x	Database relazionale
InnoDB	-	Storage engine

3.4. DevOps Stack

Tecnologia	Versione	Utilizzo
Docker	Latest	Containerizzazione
Docker Compose	Latest	Orchestrazione container
NGINX	Alpine	Web server frontend
Git	Latest	Version control

4. Componenti Backend

4.1. FastAPI Application Structure

```
backend/
├── main.py .....Entry point API & endpoints
├── models.py .....SQLAlchemy models (tabelle DB)
├── schemas.py .....Pydantic schemas (validazione)
├── auth.py .....Autenticazione & JWT logic
├── database.py .....Database connection & session
├── requirements.txt .....Python dependencies
├── Dockerfile .....Container definition
└── comuni.json .....Data comuni italiani
```

4.2. API Endpoints Principali

Autenticazione

- POST /auth/login — Login utente
- POST /auth/refresh — Refresh token
- POST /auth/logout — Logout

Utenti

- GET /users — Lista utenti
- POST /users — Crea utente
- PUT /users/{id} — Aggiorna utente
- DELETE /users/{id} — Elimina utente

Corsi

- GET /corsi — Lista corsi master
- POST /corsi — Crea corso
- PUT /corsi/{id} — Aggiorna corso
- DELETE /corsi/{id} — Elimina corso

Calendario

- GET /calendario — Lista lezioni
- POST /calendario — Crea lezione
- PUT /calendario/{id} — Aggiorna lezione
- DELETE /calendario/{id} — Elimina lezione

Presenze

- GET /presenze — Lista presenze
- POST /presenze — Crea presenza
- PUT /presenze/{id} — Aggiorna presenza
- POST /presenze/batch — Batch presenze

Studente (QR Code)

- GET /studente/lezione-oggi — Verifica lezioni oggi
- POST /studente/genera-qr — Genera QR code
- POST /studente/timbra-qr — Registra timbratura

4.3. Business Logic Layer

4.3.1. Authentication System

- **JWT Access Token:** Scadenza 15 minuti.
- **Refresh Token:** Cookie HttpOnly, scadenza 7 giorni.
- **Password Hashing:** bcrypt con salt.
- **Role-based Access:** Studente, Segreteria, Docente.

4.3.2. Validation Layer

- **Pydantic Schemas:** Validazione automatica input.
- **SQLAlchemy Constraints:** Validazione a livello database.
- **Custom Validators:** Logica business specifica.

5. Componenti Frontend

5.1. Frontend Structure

```
frontend/
├── index.html .....Landing page
├── login.html .....Login page
├── shared/ .....Componenti condivisi
│   ├── styles.css .....CSS globale
│   └── auth.js .....Auth & API client
├── segreteria/ .....Area segreteria
│   ├── dashboard.html, calendario.html, timbrature.html,
│   ├── corsi.html, aule.html, [altre pagine...]
│   └── studente/ .....Area studente
│       ├── dashboard.html, lezioni.html, timbrature.html,
│       ├── app.js, studente.css
│       └── assets/ .....Risorse statiche
│           ├── bootstrap.min.css, bootstrap-icons.css,
│           ├── bootstrap.bundle.min.js, img/
│           └── Dockerfile .....Container definition
```

5.2. Frontend Architecture

5.2.1. CSS Architecture

- **Shared Styles:** Global CSS utility classes.
- **Segreteria CSS:** Specifico area segreteria.
- **Studente CSS:** Specifico area studente.
- **Mobile-First:** Responsive design priority.
- **Design System:** Colori, tipografia, componenti riutilizzabili.

5.2.2. JavaScript Architecture

- **Modular Pattern:** Separazione per area funzionale.
- **API Client:** Centralizzato in `auth.js`.
- **State Management:** Gestione stato locale.
- **Error Handling:** Toast notifications.
- **SPA Routing:** Navigazione client-side.

5.2.3. Responsive Design

- **Breakpoints:** 576px (mobile), 768px (tablet), 992px (desktop).
- **Mobile Navigation:** Bottom bar per studenti.
- **Touch Optimization:** Target minimi 44px per iOS.
- **Performance:** Lazy loading, CSS optimization.

6. Database e Modelli Dati

6.1. Schema Database - Tabelle Principali

utenti

id_utente (PK), Nome, Cognome, Email, Password_Hash, Ruolo (studente/segreteria/docente), Telefono, Data_Nascita, Citta_Nascita, Email_Parenti, Telefono_Parenti.

corsi

id_corso (PK), Nome, Descrizione, Area_Tematica, Durata_Ore.

corsi_attivi

id_corso_attivo (PK), id_corso (FK), data_inizio, data_fine, archiviato.

calendario

id (PK), data, ora_inizio, ora_fine, id_modulo (FK), id_utente (FK - docente), id_corso_attivo (FK), note.

presenze

id_presenza (PK), id_utente (FK), id_corso_attivo (FK), data_presenza, ora_ingresso, ora_uscita, note.

6.2. Relazioni Database

- `corsi` → `corsi_attivi`: One-to-Many
- `corsi_attivi` → `calendario`: One-to-Many
- `corsi_attivi` → `utenti_corsi_attivi`: One-to-Many
- `utenti` → `presenze`: One-to-Many
- `utenti` → `calendario`: One-to-Many (docenti)

6.3. Trigger e Constraints

- **Foreign Key Constraints:** Integrità referenziale.
- **Unique Constraints:** Prevenzione duplicati.
- **Check Constraints:** Validazione dati.
- **Trigger Automatici:** Aggiornamenti timestamp.

7. Sicurezza e Autenticazione

7.1. Authentication Flow

1. User → POST /auth/login (username, password)
2. Backend → Valida credenziali
3. Backend → Genera JWT Access Token (15min)
4. Backend → Set HttpOnly Refresh Token Cookie (7giorni)
5. Frontend → Salva Access Token in localStorage
6. Richieste API → Include Authorization: Bearer {token}
7. Token scaduto → POST /auth/refresh (automatico)
8. Refresh fallito → Logout automatico

7.2. Security Measures

7.2.1. Backend Security

- **Password Hashing:** bcrypt con salt rounds.
- **JWT Validation:** Signature verification.
- **CORS Configuration:** Origins whitelist.
- **SQL Injection Prevention:** Parameterized queries.
- **XSS Protection:** Input sanitization.
- **Rate Limiting:** (implementabile).

7.2.2. Frontend Security

- **HttpOnly Cookies:** Refresh token protetto.
- **Secure Flag:** HTTPS only in produzione.
- **SameSite Policy:** CSRF protection.
- **Content Security Policy:** (implementabile).
- **Input Validation:** Client-side + server-side.

7.2.3. QR Code Security

- **Time-limited Tokens:** Validità 5 minuti.
- **User-specific QR:** Binding allo studente.
- **Date Validation:** Solo lezioni del giorno.
- **Course Verification:** Solo corsi iscritti.
- **One-time Use:** Token monouso.

8. DevOps e Deployment

8.1. Docker Configuration

8.1.1. Backend Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.1.2. Frontend Dockerfile

```
FROM nginx:alpine
COPY . /usr/share/nginx/html/
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

8.1.3. Docker Compose

```
services:
  backend:
    build: ./backend
    ports: ["8000:8000"]
    environment:
      - DATABASE_URL=mysql+pymysql://...
    volumes:
      - ./backend:/app
    command: uvicorn main:app --reload
  frontend:
    build: ./frontend
    ports: ["3000:80"]
    volumes:
      - ./frontend:/usr/share/nginx/html
    depends_on:
      - backend
```

8.2. Deployment Strategy

8.2.1. Development

- **Hot Reload:** --reload flag per sviluppo.
- **Volume Mounts:** Live code editing.
- **Local Database:** MySQL locale o remote.

8.2.2. Production

- **Multi-stage Builds:** Ottimizzazione immagini.
- **Environment Variables:** Configurazione esterna.
- **Reverse Proxy:** NGINX production configuration.

- **SSL/TLS:** HTTPS con Let's Encrypt.
- **Health Checks:** Container monitoring.
- **Logging:** Centralized logging.
- **Backup Strategy:** Database dumps regolari.

9. Funzionalità Principali

9.1. Area Segreteria

9.1.1. Gestione Corsi

- CRUD completo corsi master.
- Gestione aree tematiche.
- Definizione durata ore.
- Archiviazione corsi.

9.1.2. Gestione Edizioni

- Creazione edizioni da corsi master.
- Definizione periodi didattici.
- Gestione stato archiviazione.
- Assegnazione studenti.

9.1.3. Calendario Didattico

- Creazione lezioni singole.
- Generazione settimane complete.
- Assegnazione docenti e moduli.
- Gestione orari differenziati.
- Budget ore per unità formative.

9.1.4. Gestione Presenze

- Registro presenze giornaliero.
- Appello rapido per classe.
- Statistiche presenze/assenze.
- Export dati presenze.

9.2. Area Studente

9.2.1. Dashboard

- Panoramica corsi ed edizioni.
- Statistiche presenze.

- Lezioni del giorno.
- Ultima timbratura.

9.2.2. Lezioni

- Calendario personale interattivo.
- Dettagli lezioni (docente, modulo, orario).
- Filtri per periodo.

9.2.3. Timbrature QR Code

- Verifica lezioni del giorno.
- Generazione QR code timbratura.
- Timbratura ingresso/uscita.
- Storico presenze.

9.3. Sistema Timbratura QR Code

9.3.1. Workflow

1. **Verifica Lezione:** Sistema controlla lezioni odierne.
2. **Generazione QR:** QR code con dati crittografati.
3. **Validazione:** Scansione e validazione token.
4. **Registrazione:** Salvataggio automatico presenze.
5. **Feedback:** Conferma immediata all'utente.

9.3.2. Sicurezza QR

- Validità temporale (5 minuti).
- Binding utente-specifico.
- Verifica corso iscrizione.
- Anti-replay protection.

10. Performance e Scalabilità

10.1. Performance Optimization

10.1.1. Frontend

- **Minification:** CSS/JS compressi.
- **Lazy Loading:** Caricamento on-demand.
- **Image Optimization:** Formati WebP, lazy load.
- **Caching Strategy:** Browser caching headers.
- **CDN Integration:** (implementabile).

10.1.2. Backend

- **Database Indexing:** Indici su colonne frequenti.
- **Query Optimization:** N+1 prevention.
- **Connection Pooling:** SQLAlchemy connection pool.
- **Async Operations:** Operazioni I/O asincrone.
- **Response Compression:** Gzip encoding.

10.1.3. Database

- **Index Strategy:** Indici su foreign keys.
- **Query Optimization:** EXPLAIN analysis.
- **Connection Pooling:** Gestione connessioni.
- **Partitioning:** (implementabile per grandi dataset).

10.2. Scalability Strategy

10.2.1. Horizontal Scaling

- **Load Balancing:** NGINX load balancer.
- **Multiple Backend Instances:** Uvicorn workers.
- **Database Replication:** Read replicas.
- **Caching Layer:** Redis integration.

10.2.2. Vertical Scaling

- **Resource Allocation:** CPU/RAM optimization.
- **Database Tuning:** MySQL configuration.
- **Connection Limits:** Optimized pool sizes.

11. Conclusioni e Sviluppi Futuri

11.1. Conclusioni

ITS Manager rappresenta una soluzione completa e moderna per la gestione didattica di istituti ITS, combinando:

- **Architettura moderna:** Microservizi containerizzati.
- **Stack tecnologico avanzato:** FastAPI, Python, NGINX.
- **Sicurezza robusta:** JWT, encryption, validation.
- **UX eccellente:** Responsive design, mobile-first.
- **Scalabilità:** Progettato per la crescita.
- **Manutenibilità:** Codice modulare e documentato.

Il sistema è pronto per deployment in produzione e può essere facilmente esteso con nuove funzionalità secondo le esigenze dell'istituto.

11.2. Future Enhancements

11.2.1. Planned Features

- **Mobile App:** Native iOS/Android.
- **Push Notifications:** Lezioni e promemoria.
- **Advanced Analytics:** Dashboard predittive.
- **Integration Systems:** ERP/CRM integration.
- **Offline Mode:** PWA capabilities.
- **AI Features:** Pattern recognition presenze.

11.2.2. Technical Improvements

- **Microservices Architecture:** Service separation.
- **Event-Driven:** Message queue integration.
- **Advanced Caching:** Redis implementation.
- **API Versioning:** Backward compatibility.
- **Testing Suite:** Unit/integration tests.
- **CI/CD Pipeline:** Automated deployment.

Documento Versione: 1.0		Data: 29 Settembre 2026
Autore: Team di Sviluppo ITS Manager		Stato: Completo per Presentazione Aziendale